

Laboratory Work 4: Mobile Programming

Testing Common Flutter Widgets for Mobile (iOS / Android)

Author: Mukhammadali Khayotov | **Duration:** 90 Minutes

1 Overview & Objectives

In this laboratory work, students will explore essential Flutter UI widgets for cross-platform mobile development (iOS and Android). You will analyze a sample Flutter implementation featuring basic layout, icon, and image components, and then complete hands-on implementation exercises across 10 distinct widget categories.

2 Core Example: Icon & Image Widgets in Flutter

The following Flutter code demonstrates how to use the `Icon` widget alongside an `Image.asset` widget in a stateful mobile layout.

```
1 import 'package:flutter/material.dart';
2
3 class MediaProfileCard extends StatefulWidget {
4   const MediaProfileCard({Key? key}) : super(key: key);
5
6   @override
7   State<MediaProfileCard> createState() => _MediaProfileCardState();
8 }
9
10 class _MediaProfileCardState extends State<MediaProfileCard> {
11   bool isFavorite = false;
12
13   @override
14   Widget build(BuildContext context) {
15     return Card(
16       margin: const EdgeInsets.all(16.0),
17       child: Padding(
18         padding: const EdgeInsets.all(16.0),
19         child: Column(
20           mainAxisAlignment: MainAxisAlignment.min,
21           children: [
22             // Local asset image display
23             // NOTE: Ensure you add your image file (e.g., assets/
profile.png)
24             // to your pubspec.yaml file under the assets section!
25             ClipRRect(
26               borderRadius: BorderRadius.circular(12.0),
27               child: Image.asset(
28                 'assets/profile.png', // Replace with your own image
path
29                 width: 120,
30                 height: 120,
31                 fit: BoxFit.cover,
32                 errorBuilder: (context, error, stackTrace) {
33                   return Container(
34                     width: 120,
35                     height: 120,
```

```

36         color: Colors.grey[300],
37         child: const Icon(Icons.broken_image, size: 40),
38       );
39     },
40   ),
41 ),
42 const SizedBox(height: 12.0),
43 const Text(
44   'User Profile Card',
45   style: TextStyle(fontSize: 18, fontWeight: FontWeight.
bold),
46 ),
47 const SizedBox(height: 8.0),
48 // Interactive Icon widget
49 IconButton(
50   icon: Icon(
51     isFavorite ? Icons.favorite : Icons.favorite_border,
52     color: isFavorite ? Colors.red : Colors.grey,
53     size: 28.0,
54   ),
55   onPressed: () {
56     setState(() {
57       isFavorite = !isFavorite;
58     });
59   },
60 ),
61 ],
62 ),
63 ),
64 );
65 }
66 }

```

3 Practical Widget Tasks & Exercises

Implement the following 10 widget tasks in your Flutter project targeting mobile viewports.

3.1 Task 1: Selection Controls (Checkbox & Switch)

Objective: Master stateful toggle components.

- **Exercise 1.1:** Create a settings screen containing a `SwitchListTile` for "Dark Mode" and a `CheckboxListTile` for "Agree to Terms".
- **Exercise 1.2:** Ensure toggling "Agree to Terms" enables or disables an `ElevatedButton` below it.

3.2 Task 2: Input Fields (TextField & TextFormField)

Objective: Capture and validate mobile user inputs.

- **Exercise 2.1:** Build a login form using `TextFormField` with obscure text toggling for a password field.
- **Exercise 2.2:** Add built-in validation checking that the email field contains an @ symbol before form submission.

3.3 Task 3: Buttons & Action Items (FloatingActionButton & ElevatedButton)

Objective: Handle user tap events and primary screen actions.

- **Exercise 3.1:** Create a counter screen featuring a `FloatingActionButton` in the bottom corner that increments a counter.
- **Exercise 3.2:** Add a secondary `OutlinedButton` that resets the counter value back to 0.

3.4 Task 4: Indicators & Feedback (CircularProgressIndicator & SnackBar)

Objective: Provide immediate feedback for asynchronous operations.

- **Exercise 4.1:** Implement a button that shows a centered `CircularProgressIndicator` for 3 seconds when tapped.
- **Exercise 4.2:** Display a `SnackBar` notification at the bottom of the screen with an "Undo" action upon completion.

3.5 Task 5: Dialogs & Modals (AlertDialog & showModalBottomSheet)

Objective: Construct overlay interfaces for critical user interactions.

- **Exercise 5.1:** Implement an `AlertDialog` asking confirmation to delete an item with "Cancel" and "Delete" actions.
- **Exercise 5.2:** Create a bottom action sheet using `showModalBottomSheet` containing share options (`ListTile` items).

3.6 Task 6: Sliders & Pickers (Slider & showDatePicker)

Objective: Capture quantitative range selections and calendar inputs.

- **Exercise 6.1:** Build a custom volume control screen using a `Slider` widget that dynamically updates a displayed percentage text.
- **Exercise 6.2:** Add a button that opens a native date picker via `showDatePicker` and displays the formatted date.

3.7 Task 7: Scrollable Collections (ListView.builder & ListTile)

Objective: Display dynamic, lazy-loaded list structures.

- **Exercise 7.1:** Generate a dynamic list of 20 items using `ListView.builder` where each entry is rendered as a `ListTile`.
- **Exercise 7.2:** Implement swipe-to-dismiss functionality for list items using the `Dismissible` widget wrapper.

3.8 Task 8: Grid Displays (GridView.count)

Objective: Layout spatial collections for mobile screens.

- **Exercise 8.1:** Create a 2-column image gallery using `GridView.count` with cross-axis spacing.
- **Exercise 8.2:** Wrap each grid item in an `InkWell` or `GestureDetector` to show a full-screen preview when tapped.

3.9 Task 9: Navigation Controls (BottomNavigationBar & TabBar)

Objective: Structure multi-screen navigation flows.

- **Exercise 9.1:** Construct a 3-tab layout using `BottomNavigationBar` that switches displayed views dynamically.
- **Exercise 9.2:** Implement a top tab interface using `TabBar` and `TabBarView` inside an `AppBar`.

3.10 Task 10: Structural Containers (Card & ExpansionTile)

Objective: Group related content using elevation and collapsible containers.

- **Exercise 10.1:** Design an information card containing a header, subtitle, leading `Icon`, and trailing action button inside a `Card`.
- **Exercise 10.2:** Create an FAQ screen using multiple `ExpansionTile` widgets that expand/collapse detailed text answers when tapped.